Shafiuddin Sayem

Final Assignment

IT-21010

1. Shor's algorithm, developed by mathematician Peter Shor in 1994, poses a significant threat to widely used cryptography systems, particularly RSA and Elliptic Curve Cryptography (ECC). Both RSA and ECC relay on the computational difficulty of specific mathematical problems. RSA is based on the difficulty of factoring large integers while ECC depends on the difficulty of solving the elliptic curve discrete algorithm problem. Classical computers are unable to solve these problems efficiently, which is why these system are considered secure today. However, shor's algorithm, when run on a sufficiently powerful computer, can solve both of these problems in polynomial time. In summary it is undermines the core assumptions that make RSA & ECC secure. Its potential realization threatens the integrity, confidentiality and trust of the entire digital ecosystem, necessitating a swift transition to quantum-safe cryptography.

IT-2010

Q2] Quantum Key Distribution (QKD) is a revolutionary approach to secure communication that uses principles of quantum mechanics to distribute encryption keys between parties in a provably secure manner. Unlike classical public-key encryption, which relies on computational hardness assumptions QKD derives its security from the laws of physics specifically the behavior of quantum particles such as photons. The primary role of QKD in future cryptographic system is to enable information-theoretically secure key exchange, making it immune to attacks from both classical & quantum computers. As quantum computing threatens traditional cryptography QKD offers a future-proof method of establishing secure communication channels. The key differences are

i) Security Basis: It is based on quantum physics

ii) Resistance to Quantum attacks: QKD is inherently secure against quantum attacks.

iii) Key Distribution: QKD requires special quantum channels.

iv) Scalability: classical are easier to scale globally while QKD faces challenges in long-distance transmission

IT-2|0|0

B) Lattice-based cryptography and traditional number-theoretic cryptography represent two fundamentally different approaches to securing data. These differences become especially significant in the face of emerging quantum computing technologies.

i) Underlying Hard Problems: RSA/ECC based on number-theoretic problems such as integer factorization and discrete logarithm problem

ii) Quantum Resistance: RSA/ECC vulnerable to shors algorithm which can efficiently factor large numbers lattice-based cryptography currently considered quantum resistant as no efficient quantum algorithm needed

iii) Security Guarantees: RSA/ECC it is based on unproven mathematical assumptions. Lattice-based offers strong worst case to average case hardness.

iv) Performance & Efficiency: RSA/ECC mature and well optimized. Lattice based requires large key

v) Applications: RSA/ECC used in SSL/TLS digital signatures, encrypted email. Lattice based supports advanced cryptographic primitives like fully homomorphic encryption.

6/09/25 IT-21010

Q4. Custom PRNG Using Time & Seed (Python code below)

```
import time
class CustomPRNG:
  def __init__(self, seed=None):
    if seed is None:
      seed = int(time.time_ns())
    self.seed = seed
    self.modulus = 2**32
    self.a = 1664525
    self.c = 1013904223
    self.state = seed
    print(f"[INFO] PRNG initialized with seed: {self.seed}")
  def next(self):
    self.state = (self.a * self.state + self.c) % self.modulus
    return self.state
  def random(self):
    return self.next() / self.modulus
if __name__ == "__main__":
  custom_seed = int(time.time_ns() % 123456789)
  prng = CustomPRNG(seed=custom_seed)
  print("\n Generating 5 pseudo-random numbers...")
  for i in range(5):
    print(f"Random #{i+1}: {prng.random()}")
```

IT-2010

95] Prime less than 50 python implementation:

```
def sieve_of_eratosthenes(n):
    is_prime = [True] * n
    is_prime[0:2] = [False, False]
    for p in range(2, int(n**0.5)+1):
        if is_prime[p]:
            for i in range(p*p, n, p):
                is_prime[i] = False
    primes = [i for i, prime in enumerate(is_prime) if prime]
    return primes
primes_below_50 = sieve_of_eratosthenes(50)
print("Prime numbers less than 50:", primes_below_50)
```

output: prime numbers less than 50:

[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

Time Complexity comparison:

Sieve of Eratosthenes O(n log log n)

Efficient for generating all primes upto n

Trial Division O(n√n) or O(n²)

checks each number of for divisibility upto √n

IT-2|0|0

Q6| A Carmichael number is a special type of composite number n that fails Fermat's primality test. That is A composite number n is called a Carmichael number if $a^{n-1} \equiv 1 \bmod n$ for all integers a such that $\gcd(a,n)=1$

Necessary & Sufficient Conditions:

n is a carmichael number if & only if:

i) n is square free and ii) For every prime divisor p of n, it holds that $p-1 \mid n-1$

Now For n = 561

Prime factorization

$561 = 3 \times 11 \times 17$

all primes are distinct

check condition for each p|561

p = 3: 3-1 = 2, and 2|560

p = 11: 11-1 = 10 and 10|560

p = 17: 17-1 = 16 and 16|560

It is a carmichael number

For n = 1729

Prime Factorization

$1729 = 7 \times 13 \times 19$

7-1 = 6, and 6|1728

13-1 = 12, and 12|1728

19-1 = 18, and 18|1728

So, It is a carmichael number

For n = 1105

prime factorization $1105 = 5 \times 13 \times 17$

check conditions:

5-1 = 4, and 4|1104

13-1 = 12 and 12|1104

17-1 = 16 and 16|1104

It is a carmichael number

IT-21010

Z_{11} is ring because Z_{11} is the set of integers modulo 11 $Z_{11} = \{0,1,2,\ldots,10\}$ addition & multiplication are defined modulo 11. A ring requires: $(Z_{11}, +)$ is an Abelian group: closure, associativity, identity(0), inverse and commutativity are all satisfied. $(Z_{11}, \cdot)$ is closed and associative. Distributive property holds $a.(b+c) \equiv a.b + a.c \pmod{11}$.

$(Z_{37}, +)$ is an Abelian group. Because it consists of integers modulo 37; $Z_{37} = \{0,1,\ldots,36\}$

Operation: Addition modulo 37

Group axioms: closure, Associativity, Identity, Inverse: $\forall a \in Z_{37}, \exists -a \bmod 37$, commutativity.

$(Z_{35}^{*}, \times)$ It is also Abelian group. It is the set of units modulo 35 under multiplication

$$Z_{35}^{*} = \{ a \in Z_{35} \mid \gcd(a, 35) = 1 \}$$

$35 = 5 \times 7 \Rightarrow$ No 1 prime, so not all elements are invertible

The units are $Z_{35}^{*} = \{1, 2, 3, \ldots, 34\}$

It is closure, associativity, identity, inverse and commutative

Q28] To find the remainder we compute $-52 \bmod 31$. Modulo always returns a non-negative remainder between 0 and $n-1$, we want to find a number r such that $-52 \equiv r \bmod 31$ where $0 \le r < 31$

$-52 + 31 = -21$ (still negative)

$-21 + 31 = 10$ (between 0 & 31)

$-52 \equiv 10 \bmod 31$

So, our remainder is 10

Q29] To find the multiplicative inverse of 7 mod 26, we want to find an integer x such that $7x \equiv 1 \bmod 26$

This is equivalent solving $7x + 26y = 1$

Start from $1 = 5 - 2 \cdot 2$

but from above $2 = 7 - 1 \cdot 5$ so

$$1 = 5 - 2(7 - 1 \cdot 5) = 5 - 2 \cdot 7 + 2 \cdot 5 = 3 \cdot 5 - 2 \cdot 7$$

now replace $5 = 26 - 3 \cdot 7$

$$1 = 3(26 - 3 \cdot 7) - 2 \cdot 7 = 3 \cdot 26 - 9 \cdot 7 - 2 \cdot 7 = 3 \cdot 26 - 11 \cdot 7$$

So $1 = 3 \cdot 26 - 11 \cdot 7$

final answer $-11 \cdot 7 + 3 \cdot 26 = 1 \Rightarrow x = -11$

$-11 \equiv 15 \bmod 26$

So the multiplicative inverse of 7 mod 26 is (15)

IT-2010

2(b) First Multiply $-8 \times 5 = -70$

Reduce Modulo 17 we want $-70 \bmod 17$,
To get a positive remainder, we can add 17
repeatedly until the result is in the range

$0 \le r < 17$ $\quad -70 + 17 = -23$

$-23 + 17 = -6$

$-6 + 17 = (11)$

So our result is 11

The general rule is $(a.b) \bmod m$
When dealing with negative numbers modulo m,
convert them first using

$a \bmod m = (a+m) \bmod m$ (if $a < 0$)

2(c) Statement of Bezout's Theorem: Let a & b be Integers
not both zero. Then there exist integers x and y such that

$ax + by = \gcd(a, b)$

If $\gcd(a, m) = 1$, then Bezout's identity tells us there
exists an x such that

$ax + my = 1 \Rightarrow ax \equiv 1 \pmod m$

IT-21010

We need to solve $97x \equiv 1 \bmod 385$

which means: $97x + 385y = 1$

we compute gcd (385, 97):

$385 = 3 \cdot 97 + 94$

$97 = 1 \cdot 94 + 3$

$94 = 31 \cdot 3 + 1$

$3 = 3 \cdot 1 + 0$ ∴ gcd(97, 385) = 1 = inverse exist

Now Extended Euclidean Algorithm

Starts from: $1 = 94 - 31 \cdot 3$

substitute $3 = 97 - 1 \cdot 94$

$1 = 94 - 31(97 - 1 \cdot 94) = 94 - 31 \cdot 97 + 31 \cdot 94$

$= 32 \cdot 94 - 31 \cdot 97$

Now substitute $94 = 385 - 3 \cdot 97$

$1 = 32(385 - 3 \cdot 97) - 3 \cdot 97$

$= 32 \cdot 385 - 96 \cdot 97 - 31 \cdot 97$

$= 32 \cdot 385 - 127 \cdot 97$

$1 = 32 \cdot 385 - 127 \cdot 97$

So our final answer $97(-127) + 385 \cdot 32 = 1 \Rightarrow -127$ is the inverse of 97 mod 385

We want the positive inverse, so reduce

$-127 \bmod 385 = 385 - 127 = 258$

IT-2/010

12] Bezouds identity (existence): Let a, b be integers not both zero and let d = gcd(a,b). Run the Euclidean algorithm to compute d

$$a = q_1 b + r_1, \quad 0 \le r_1 < b.$$
$$b = q_2 r_1 + r_2$$
$$r_1 = q_3 r_2 + r_3$$
$$\vdots$$
$$r_{k-2} = q_k r_{k-1} + r_k$$
$$r_{k-1} = q_{k+1} r_k + 0$$

So $r_k = d$. Each remainder r_i is an integer linear combination of a and b because $r_1 = a - q_1 b$ and thereafter $r_i = r_{i-1} - q_{i+1} r_i$. By back substitution we express r_k as $r_k = xa + yb$ for some integers x, y. Hence d is an integer linear combination of a and b. Since any common divisor of a and b divides every linear combination, d is the smallest positive such combination and we have Bezouds identity $ca + cb = \gcd(a,b)$.

Find x with $93x \equiv 1 \pmod{290}$ we need an integer x with $93x + 290y = 1$. Use the Euclidean algorithm

$$290 = 5 \cdot 93 + 25 \qquad 93 = 1 \cdot 3 + 1$$
$$93 = 1 \cdot 25 + 18 \qquad 3 = 1 \cdot 3 + 0$$
$$25 = 1 \cdot 18 + 7$$
$$18 = 2 \cdot 7 + 4$$
$$7 = 1 \cdot 4 + 3$$

IT-2010

Back-substitute to express 1 as a combination of 93 & 290

$1 = 9 - 1 \cdot 3$

$= 9 - 1(7 - 1 \cdot 9) = 2 \cdot 9 - 1 \cdot 7$

$= 2(18 - 2 \cdot 7) - 1 \cdot 7 = 2 \cdot 18 - 5 \cdot 7$

$= 2 \cdot 18 - 5(25 - 1 \cdot 18) = 7 \cdot 18 - 5 \cdot 25$

$= 7(93 - 1 \cdot 25) - 5 \cdot 25 = 7 \cdot 93 - 12 \cdot 25$

$= 7 \cdot 93 - 12(290 - 3 \cdot 93) = (7 + 60) \cdot 93 - 12 \cdot 290$

$= 67 \cdot 93 - 12 \cdot 290$

So $67 \cdot 93 + (-12) \cdot 290 = 1$. Thus $x = 67$ is a solution and this gives the multiplicative inverse of 93 modulo 29

[B] Fermat's Little Theorem is: If p is prime and a is an integer with $\gcd(a, p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$

Consider the nonzero residue classes modulo p: $\{1, 2, \ldots, p-1\}$. Multiplying each element by a (with $\gcd(a, p) = 1$) permutes this set modulo p. So $a \cdot 1, a \cdot 2, \ldots, a \cdot (p-1)$ is congruent modulo p to some permutation of $1, 2, \ldots, p-1$. Taking the product of the terms in both lists gives

$$a^{p-1} \cdot (1 \cdot 2 \cdots (p-1)) \equiv 1 \cdot 2 \cdots (p-1) \pmod{p}$$

Since $1 \cdot 2 \cdots (p-1)$ is not divisible by p, we may cancel it in the field $\mathbb{Z}/p\mathbb{Z}$, yielding $a^{p-1} \equiv 1 \pmod{p}$.

In 561 prime based on this test?
factor $561 = 3 \cdot 11 \cdot 17$. For any a with $\gcd(a, 561) = 1$, by Fermat's theorem we have $a^2 \equiv 1 \pmod 3$, $a^{10} \equiv 1 \pmod{11}$, $a^{16} \equiv 1 \pmod{17}$.
Since $\text{lcm}(2, 10, 16) = 80$ & 560 is a multiple of each of 2, 10, 16 (indeed $560 = 7 \cdot 80$) we get $a^{560} \equiv 1$ modulo each of 3, 11, 17. By the chinese Remainder Theorem
$a^{560} \equiv 1 \pmod{561}$, now,
Evaluate $5^{123} \bmod 175$. Note $175 = 25 \cdot 7$ because $\gcd(5, 7) = 5 \neq 1$. Fermat's theorem doesn't apply directly to modulus 175.
1. Modulo 25 for $n \geq 2$, $5^n = 25 \equiv 0 \pmod{25}$ Hence for any exp ≥ 2
2. Modulo 7. $5^{123} \equiv 0 \pmod{25}$
Here $\gcd(5, 7) = 1$, so we can use Fermat $5^6 \equiv 1 \pmod 7$
$123 \equiv 3 \pmod 6$ (since $123 = 6 \cdot 20 + 3$) $5^{123} \equiv 5^3 \pmod 7$.
3. Combine CRT to find x. $x \equiv 0 \pmod{25}$, $x \equiv 6 \pmod 7$
$k \equiv 2$, $6 \equiv 12 \equiv 5 \pmod 7$
So $k = 5 + 7l$ and $x = 25k = 25 \cdot 5 + 25 \cdot 7l = 125 + 175l$
The least residue modulo 175 is $x \equiv 125 \pmod{175}$
So the answer is $5^{123} \equiv 125 \pmod{175}$

IT-2010

2[9] Statement: let $n_1, n_2, \ldots, n_k$ be pairwise coprime positive integers. For any integers $a_1, a_2, \ldots, a_k$ the system $x \equiv a_i \pmod{n_i}$ has a solution x. Moreover all solutions are congruent modulo $N = n_1 n_2 \cdots n_k$. The solution is unique modulo N. Now we have

$$\begin{cases} x \equiv 2 \pmod 3 \\ x \equiv 3 \pmod 5 \\ x \equiv 2 \pmod 7 \end{cases}$$

Here $n_1 = 3$, $n_2 = 5$, $n_3 = 7$. They are pairwise coprime. Compute $N = 3 \cdot 5 \cdot 7 = 105$

Compute N_i:

$N_1 = 105/3 = 35$, $N_2 = 105/5 = 21$, $N_3 = 105/7 = 15$.

Find inverses y_i with $N_i y_i \equiv 1 \pmod{n_i}$:

For $n_1 = 3$: $35 \equiv 2 \pmod 3$. Inverse of 2 mod 3 is 2 (since $2 \cdot 2 = 4 \equiv 1 \pmod 3$)

For $n_2 = 5$: $21 \equiv 1 \pmod 5$. Inverse of 1 is 1. So $y_2 = 1$

For $n_3 = 7$: $15 \equiv 1 \pmod 7$. Inverse is 1. So $y_3 = 1$

Form the solution: $x = \sum_{i=1}^{3} a_i N_i y_i = 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1$

calculate: $x = 2 \cdot 70 + 63 + 30 = 140 + 63 + 30 = 233$

Reduce modulo $N = 105$

$$233 \equiv 233 - 2 \cdot 105 = 233 - 210 = 23 \pmod{105}$$

So we get all of this congruence to hold

$23 \bmod 3 = 2$

$23 \bmod 5 = 3$

$23 \bmod 7 = 2$

So our final answer is $x \equiv 23 \pmod{105}$

IT-2010

[5] The CIA triad is a foundational model in Information Security that represents the three key principles needed to build and maintain secure systems.

i) Confidentiality: Ensures the sensitive information is accessible only to authorized users and not disclosed to unauthorized individuals. Protects privacy and prevents data breaches. Encryption, access controls and data classifications are technologies.

ii) Integrity: Guarantees that info remains accurate complete and unaltered except by authorized changes prevents unauthorized modification or corruption of data. checksums hashing digital signatures version control. Verifying file integrity using a hash value after downloading software.

iii) Availability: Ensures that info and systems are accessible to authorized users when needed. Prevents disruptions in service that could cause business or operational failures. Techniques are Redundancy, load balancing regular backups disaster recovery. Available resources even during high traffic using scalable cloud infrastructure.

IT-2010

16] Difference between Steganography & Cryptography

Cryptography	Steganography
i) Scrambles the message so that even if intercepted. It can't be understood without a key	i) Hides the existence of the message so that no one knows it's there
ii) The msg is visible but unreadable without decryption	ii) Msg is invisible hidden inside other harmless-looking data
iii) Mathematical algorithms	iii) Concealment methods
iv) Example: HELLO → JGNNQ	iv) HELLO hidden inside a photo

Common Steganography Techniques for Digital Media

A] Image Steganography: LSB Insertion, Palette-based Hiding, Transform Domain Techniques

B] Audio Steganography: LSB coding, Phase coding, Echo Hiding.

C] Video Steganography: Frame-by-Frame LSB, Motion Vector Alteration

D] Text Steganography: Whitespace Manipulation, Font or Character substitution

IT-2101

[7] 1] Phishing:
Method → A social engineering attack that tricks users into revealing sensitive data. Usually carried out via fraudulent emails, fake websites, text msg or calls that appear legitimate.
Impact → compromise confidentiality by stealing personal or organizational data, can lead to identity theft, financial loss, and unauthorized account access.

2] Malware:
Method → Malicious software designed to infiltrate and damage systems. Spread via infected files, malicious downloads, email attachments.
Impact → can compromise confidentiality, integrity

3] Denial-of-Service (DoS):
Method → Floods a server, network, or application with excessive requests, making it unavailable to legitimate users.
Impact → Mainly affects availability of service, causes downtime, loss of business & reputational damage.

IT-21010

18] The General Data Protection Regulation (GDPR) is a legal framework in the European Union designed to strengthen data privacy, data security and user rights. Although GDPR is not a direct cyber security tool, it helps mitigate cyber attacks and protect user privacy in several important ways.

1] Data Protection By Design & by Default: Organizations must integrate privacy & security measures into systems from the start. Reduces the risk of data breaches because sensitive information is better secured before attackers can reach it.

2] Strict Data Minimization & Purpose Limitation: Only collect and store the minimum data necessary for specific purposes. Even if a breach occurs, the amount of data exposed is reduced, limiting attacker gains.

3] Security Measures & Breach Notification: Implement technical and organizational measures. Faster responses help contain the attack & prevent further damage.

4] Accountability & Heavy Penalties: Organizations must be proof of compliance. The risk of huge fines motivates companies to take security seriously.

5] User Rights & Transparency: Users have rights to access, correct, delete and transfer their data. Makes it hard for attackers or malicious insiders to exploit hidden or unnecessary data stores.

IT-21010

10 | Step1: Initial permutation (IP): The 64-bit plaintext is rearranged according to a fixed table. Two 32-bit halves: L0 = left 32 bits R0 = right 32 bits

Step-2: Key Preparation: Input 64-bit key → remove 8 parity bits. 56 bit key. Perform permuted choice 1 (PC-1) to rearrange bits. For each round: Left shift C & D. (Shifts vary per round i.e. 1 or 2 bits) Combine C & D → apply permuted choice 2 (PC-2) → 48 bit subkey.

Step3: 16 Feistel Rounds: Each round takes L_{i-1}, R_{i-1} & k_i to produce L_i, R_i: Permutation (P) Rearrange the 32 bits for more diffusion: $L_i = R_{i-1}$ $R_i = L_{i-1} \oplus F(R_{i-1}, k_i)$

Step4: Final Permutation (FP): After 16 rounds, combine R16 & L16. Apply Final permutation to produce ciphertext

Example for any plaintext: P = 0x0123456789ABCDEF

key K = 0x133457799BBCDFF1

Process:

1) IP: Rearrange plaintext bits

2) Round 1: Expand R0 (32 bits) → 48 bits. XOR with K1. Pass through S-boxes → 32 bits. P-box permutation. XOR with L0

3) Repeat for 16 rounds, each time with a different key

4) FP: Final permutation gives ciphertext

Final output: Ciphertext = 0x85E813540F0AB405

IT-21010

20] We are given,
$R_0 = 0xF0F0F0F0$
$K_1 = 0x0F0F0F0F$
$L_0 = 0xAAAAAAAA$
$f(R_0, K_1)$ is defined here as the XOR $R_0 \oplus K_1$
For one DES round: $L_1 = R_0$, $R_1 = L_0 \oplus f(R_0, K_1)$
Compute f:
$f(R_0, K_1) = R_0 \oplus K_1 = 0xF0F0F0F0 \oplus 0x0F0F0F0F$
$= 0xFFFFFFFF$
Each byte: $(0xF0 \oplus 0x0F = 0xFF)$
Then $L_1 = R_0 = 0xF0F0F0F0$
$R_1 = L_0 \oplus f(R_0, K_1) = 0xAAAAAAAA \oplus 0xFFFFFFFF$
$= 0x55555555$

21] Computing AES SubBytes for the input word
$[0x23, 0xA7, 0x9C, 0x19]$
using the standard AES sbox

IT-21010

sbox = [
0x63 0x7c 0x77 0x7b 0xf2 0x6f 0xc5 0x30 0x01 0x67 0x2b 0xfe 0xd7 0xab 0x76,
0xca 0x82 0xc9 0x7d 0xfa 0x59 0x47 0xf0 0xad 0xd4 0xa2 0xaf 0x9c 0xa4 0x72 0xc0,
0xb7 0xfd 0x93 0x26 0x36 0x3f 0xf7 0xcc 0x34 0xa5 0xe5 0xf1 0x71 0xd8 0x31 0x15,
0x04 0xc7 0x23 0xc3 0x18 0x96 0x05 0x9a 0x07 0x12 0x80 0xe2 0xeb 0x27 0xb2 0x75,
0x09 0x83 0x2c 0x1a 0x1b 0x6e 0x5a 0xa0 0x52 0x3b 0xd6 0xb3 0x29 0xe3 0x2f 0x84,
0x53 0xd1 0x00 0xed 0x20 0xfc 0xb1 0x5b 0x6a 0xcb 0xbe 0x39 0x4a 0x4c 0x58 0xcf,
0xd0 0xef 0xaa 0xfb 0x43 0x4d 0x33 0x85 0x45 0xf9 0x02 0x7f 0x50 0x3c 0x9f 0xa8,
0x51 0xa3 0x40 0x8f 0x92 0x9d 0x38 0xf5 0xbc 0xb6 0xda 0x21 0x10 0xff 0xf3 0xd2,
0xcd 0x0c 0x13 0xec 0x5f 0x97 0x44 0x17 0xc4 0xa7 0x7e 0x3d 0x64 0x5d 0x19 0x73,
0xe7 0xc8 0x37 0x6d 0x8d 0xd5 0x4e 0xa9 0x6c 0x56 0xf4 0xea 0x65 0x7a 0xae 0x08,
]

input_word = [0x23, 0xA7, 0x9C, 0x19]

output = [sbox[b] for b in input_word]

print("Input word:", [hex(x) for x in input_word])

print("SubBytes Output:", [hex(x) for x in output])

Using the standard AES S-box, substitute each byte

0x23 → 0x26

0xA7 → 0x5C

0x9C → 0x29

0x19 → 0xD4

So the resulting output word after SubBytes is

[0x26, 0x5C, 0x29, 0xD4]

IT-2|010

22] The AddRoundKey step in AES is just a byte-wise XOR between the input word & the round key word. We calculate byte by byte:

1] 0x1A ⊕ 0x55

0x1A = $(00011010)_2$ 0x55 = $(01010101)_2$

XOR: $(01001111)_2$ = 0x4F

2] 0x2B ⊕ 0x66

0x2B = $(00101011)_2$ 0x66 = $(01100110)_2$

XOR: $(01001101)_2$ = 0x4D

3] 0x3C ⊕ 0x77

0x3C = $(00111100)_2$ 0x77 = $(01110111)_2$

XOR: $(01001011)_2$ = 0x4B

4] 0x9D ⊕ 0x88

0x9D = $(10011101)_2$ 0x88 = $(10001000)_2$

XOR: $(00010101)_2$ = 0x15

So the output word after AddRoundKey:

[0x4F, 0x4D, 0x4B, 0x15]

IT-21010

29) Given Matrix used by MixColumns over GF(2^8)

$$M = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

Input column (as bytes):

$$a = \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0x01 \\ 0x02 \\ 0x03 \\ 0x04 \end{bmatrix}$$

Also convert each input byte to 8-bit binary

$0x01 = 00000001_2$
$0x02 = 00000010_2$
$0x03 = 00000011_2$
$0x04 = 00000100_2$

Compute each output byte: The product is $r = M \cdot a$

$r_0 = 02 \cdot a_0 \oplus 03 \cdot a_1 \oplus 01 \cdot a_2 \oplus 01 \cdot a_3$

$02 \cdot a_0 = 02 \cdot 0x01 = 0x02 \rightarrow 00000010$

$03 \cdot a_1 = (02 \cdot 0x02) \oplus 01 \cdot 02$. Find $02 \cdot 0x02 = 0x04 (00000100)$,

then $0x04 \oplus 0x02 = 0x06 (00000110)$

$a_1 \cdot a_2 = 0x03 \rightarrow 00000011$

$a_1 \cdot a_3 = 0x04 \rightarrow 00000100$ So, $r_0 = 0x03$

computing all r_1, r_2, r_3 we get

$r_0 = 0x03$, $r_1 = 0x09$, $r_2 = 0x09$, $r_3 = 0x0A$

The transformed column after MixColumn is:

$$\begin{bmatrix} 0x03 \\ 0x09 \\ 0x09 \\ 0x0A \end{bmatrix}$$

In Binary [00000011, 00000100, 00001001, 00001010]

2) OFB mode is a stream cipher mode of AES instead of encrypting the plaintext directly with AES, it repeatedly encrypts an initial value (IV) and uses the result as a keystream to XOR with the plaintext.

How it works: Let's define:
key: AES secret key, IV: Initial vector, $E_k()$: AES encryption function with key K, $\oplus$: XOR operation

Encryption Steps:
1) Set $O_0 = IV$. 2) For block: $O_i = E_k(O_{i-1})$ (Encrypt the previous output to get the next keystream block)
$C_i = P_i \oplus O_i$ (XOR keystream with plaintext to get ciphertext)

Decryption Steps:
1) Use the same IV and key
Generate the same keystream: $O_i = E_k(O_{i-1})$
Recover plaintext: $P_i = C_i \oplus O_i$

IT-2010

Diagram: (Encryption)

IV → AES-01 - XOR with $P_1 - C_1$

↓

AES-02 - XOR with $P_2 - C_3$

↓

AES-03 - XOR with $P_3 - C_3$

- - -

- - -

(Decryption)

IV → AES-01 - XOR with $C_1 - P_1$

↓

AES-02 - XOR with $C_2 - P_2$

↓

AES-03 - XOR with $C_3 - P_3$

- -

- -

2(a) Error propagation means that a bit error in a single ciphertext block can cause multiple bits in the decrypted plaintext to be wrong. This happens because some AES modes use feedback or chaining between blocks.

Decryption process for CBC decryption is

$$P_i = D_k(C_i) \oplus C_{i-1}$$

where $D_k(C_i)$ = block decrypted with AES

C_{i-1} = previous ciphertext block (or IV for first block)

Impact: One ciphertext bit error —
Block P_i completely wrong Block P_{i+1} one bit wrong

Example: Let's say C_2 gets a 1 bit error:

$$P_2 = D_K(C_2') \oplus C_1 \text{ (garbled)}$$

$$P_3 = D_K(C_3) \oplus C_2' \text{ (1 bit error)}$$

CFB Mode (Cipher Feedback)

Decryption process (CFB with block sizes)

$$P_i = C_i \oplus E_K(C_{i-1})$$

Error effect:

If a single bit in C_i is wrong: The corresponding bit in P_i will be wrong. The error also propagates for the next few blocks depending on feedback sizes because C_i is used as input to AES encryption for future plaintext recovery.

Impact:

In CFB-128 one ciphertext bit error - current block 1 bit error, next block: full block corrupted

In smaller feedback sizes Error persists for a smaller number of bytes.

IT-2(0)0

2] Justification:

1] Parallelism: CTR mode allows encryption and decryption to be done in parallel because each block is encrypted with a unique counter value, independent for other blocks. CBC mode cannot be fully parallelized during encryption because each plaintext block must be XORed with the previous ciphertext block's encryption. ECB mode is parallelizable but insecure

2] Security: ECB mode is insecure for large files because identical plaintext blocks produce identical ciphertext blocks, revealing patterns in the data. CBC mode hides patterns but is slower for encryption due to dependency on the previous block. CTR mode hides patterns effectively and uses a keystream-like approach making it secure

3] Error Propagation: CBC mode has error propagation a single bit error in one ciphertext block affects two plaintext blocks on decryption.
CTR mode has minimal error propagation an error affects only the corresponding bits in the affected block.

27] Given that,

Message $M = 1$, Public key $e = 5$, $n = 14$

private key $d = 11$

RSA encryption formula $C = M^e \bmod n$

$C = 1^5 \bmod 14$

$C = 1 \bmod 14$

$C = 1$

∴ ciphertext $= 1$

RSA decryption formula: $M = C^d \bmod n$

$M = 1^{11} \bmod 14$

$M = 1 \bmod 14$

$M = 1$

Decrypted Message $= 1$

So our ciphertext $= 1$ and our decrypted message $= 1$

same as our original

IT-2010

28] To generate the RSA digital signature S we compute $S = [H(M)]^d \pmod{n}$

Given that

$H(M) = 5$, $d = 3$, $n = 33$.

compute 5^3 and reduce mod 33

$5^2 = 25 \pmod{33}$, $5^3 = 25 \cdot 5 = 125$

Reduce 125 mod 33. $33 \times 3 = 99$, $125 - 99 = 26$

$\therefore$ Our S is 26.

29] we can compute the public keys in Diffie-Hellmen using $\text{publickey} = g^{\text{Privatekey}} \bmod p$

Given: $p = 17$, $g = 3$, private key $a = 4$ (Alega)

private key $b = 5$ (Badal)

Alega's public key (A):

$A = g^a \bmod p = 3^4 \bmod 17$

$3^2 = 9$

$3^4 = 9^2 = 81$

$81 \bmod 17 = 81 - (17 \times 4) = 81 - 68 = 13$

$\therefore$ Alega's publeen key = 13

Date: 27/2/10

Badal's Public Key (B):

$$B = g^b \bmod p = 3^5 \bmod 17$$

$3^9 = 81 \bmod 17 = 13$ (From above)

$3^5 = 13 \times 3 = 39$

$39 \bmod 17 = 39 - (17 \times 2) = 39 - 39 = 5$

Badal's public key is 5

3) ASCII values A = 65 & B = 66

Calculate hash for "AB":

$$H("AB") = (65 + 66) \bmod 100$$

$$= 131 \bmod 100 = 31$$

Calculate hash for "BA"

$$H("BA") = (66 + 65) \bmod 100$$

$$= 131 \bmod 100 = 31$$

Both AB & BA produce the same hash value 31

This shows a hash collision as two different inputs produce the same outputs. For this hash function, the order of characters doesn't matter because it only sums ASCII values. This makes it very weak.

IT-21010

3) Formula in MAC = (Message + Secret key) mod 17

Given values Message = 15 Secret key = 7

Compute MAC MAC = (15+7) mod 17

= 22 mod 17 = 5

Mac = 5 Attacker change message to 10

The new MAC should be

MAC = (10+7) mod 17

= 17 mod 17 = 0 (but he doesn't know the key 7)

Can they forge the MAC easily? => Yes this MAC Scheme is weak. Because:

This operation is linear Mac = (msg + secret key) mod 17

If the attacker knows the original message (15) and its MAC (5), they can compute the secret key easily.

Secret key = (MAC - Message) mod 17 = (5-15) mod 17

Once they have the key, they can generate valid MAC for any modified message.

This shows poor security as simple known msg MAC pair reveals the secret key

IT-2/D/0

32) The TLS handshake is the process in which a client and a server agree on encryption parameters & securely generate a shared symmetric session key. This key is then used for fast and secure data transfer, while asymmetric cryptography is used only during the handshake to establish that key securely.

Steps in the TLS handshake

Step1: clientHello: The client sends → supported TLS version, supported cipher suites, random number, optional extensions

Step2: ServerHello: The server responds with → chosen TLS version, selected cipher suite, random number, digital certificate

Step 3 Server Authentication: The client verifies → certificate validity, domain name match, certificate hasn't expired or been revoked. This ensures the client is talking to the real server, not an impostor.

Step4 Key Exchange: There are two common methods for that (a) RSA key exchange (older) (b) Diffie-Hellman/ECDHE

Step5 Session key Derivation: Both parties now have - client Random, Server Random, pre-master secret. They feed these into a key derivation function (KDF) to produce symmetric encryption keys, message authentication keys.

Step6 Finished Messages: Both client & server send an encrypted "Finished" message using the newly derived symmetric keys. If the other side decrypts it successfully the handshake is complete.

IT-2|0|0 claude ai

B3] SSH (Secure Shell) uses a three-layer protocol stack to securely connect two systems over an insecure network.

SSH Layered Architecture

1) Transport layer (SSH-TRANS)
Establishes the secure channel between client & server handles encryption, manage integrity and server authentication
Functions: Negotiates encryption algorithms. Negotiates manage authentication codes for integrity. Authenticates the server using its host key.

2) User Authentication Layer (SSH-AUTH)
Verifies the identity of the client to the server.
Functions: Supports multiple authentication methods password authentication public key authentication
Runs after the secure channel is established in the transport layer.

3) Connection Layer (SSH-CONN)
Manages multiple logical channels over the single encrypted SSH connection. Function: Enables port forwarding. Provides interactive shell sessions. Supports file transfer. Allows multiplexing multiple streams over one TCP connection.

IT-21010

Q1| The TLS handshake is the process by which a client & a server - Agree on encryption parameters. Authenticate each other. Securely generate a shared symmetric session key for encrypted communication.

Steps in the TLS Handshake:

Step1 (ClientHello) → The client sends supported TLS version, List of supported cipher suites. Random number, optional extensions

Step2 (ServerHello) → The server replies with chosen TLS version, selected cipher suite, Random number, Server's digital certificate

Step 3 (Server Authentication) → The client verifies the server's certificate checks CA signature, matches domain value, Ensures it's valid.

Step 4 (Key Exchange) → Two main methods 1) RSA where client generates a pre-Master secret. Encrypts it with the server's public key. Sends it to the server, which decrypts it with Private key.

2) (ECDHE Preferred) → client & server exchange ephemeral public keys. Both compute the same shared secret using Diffie-Hellman math. Provides forward secrecy.

IT-2/0/0

Step5: Session key Generation → Both parties now have client random, server random, pre-master secret. They combine through a key derivation function (KDF) to generate symmetric encryption keys.

Step6: Finished Messages → Client & server send "Finished" message encrypted with the new symmetric keys. If both messages decrypt and verify correctly, the handshake is complete.

Summary flow

Client Hello → Server Hello → Certificate → Key Exchange → Session key → Finished

351 General Form of an Elliptic Curve Equation

Over a finite field F_p (where p is a prime) the general short Weierstrass form is

$$y^2 \bmod p = (x^3 + ax + b) \bmod p$$

Where $a, b \in F_p$ are constants defining the curve. The curve must satisfy the non-singularity condition

$$4a^3 + 27b^2 \neq 0 \pmod p$$

IT-2010

to ensure no cusps or self intersections. The set of points (x, y) that satisfy the equation plus a special point at infinity form an abelian group under a defined addition operation.

Elliptic curves are used in cryptography because

1) High security per bit: ECC offers the same security as RSA but with much smaller key sizes.

2) Based on the Elliptic Curve Discrete Logarithm Problem: Given two points P & $Q = kP$ on the curve, its computationally infeasible to find k (Private key) if the field size is large.

3) Efficient Computation: Smaller keys mean faster encryption, decryption and signature operations.

4) Lower Bandwidth and storage requirements: Useful for constrained devices.

$$y^2 = x^3 + ax + b \pmod{p}$$

is used because it defines a mathematical structure with a hard to reverse problem, enabling strong, efficient public-key cryptography.

AT-2|0|0

36] ECC (Elliptic Curve Cryptography) achieves the same security as RSA with a smaller key size because the mathematical Problem it's based on is much harder.

RSA vs ECC Security Basis: RSA Security is Based on the difficulty of factoring large integers. ECC security is based on the Elliptic curve Discrete logarithm problem (ECDLP). Given two points P & Q = kP on an elliptic curve, it's computationally infeasible to find k.

Hardness Difference: The best known algorithms for factoring (RSA) are much faster than the best known algorithms for solving ECDLP. This means ECC can use much smaller numbers to achieve the same security level.

Key size Comparison

Security Level	RSA Key Size	ECC Key Size
128 bit Sec	3072 bits	256 bits
192 bit Sec	7680 bits	384 bits
256 bit Sec	15360 bits	521 bits

Benefits of smaller ECC keys: Faster computations, Lower bandwidth, Less storage, Better for mobile/IoT devices.

37] We are given:

$$y^2 \equiv x^3 + 2x + 3 \pmod{97}$$

point $P = (3, 6)$.

Step 1: check LHS $y^2 = 6^2 = 36$

So LHS $\equiv 36 \pmod{97}$.

Step 2: check RHS $x^3 + 2x + 3 = 3^3 + 2(3) + 3$

$= 27 + 6 + 3 = 36$

So RHS $\equiv 36 \pmod{97}$

Step 3: Compare

L.H.S $\equiv$ R.H.S $\equiv 36 \pmod{97}$.

Yes, the point (3,6) lies on the curve

38] We are given

$p = 23$

$g = 5$

$h = 8$ (Public key $= g^x \bmod p$)

message $m = 10$

Random $k = 6$

Compute c_1 $\quad c_1 = g^k \bmod p$

$c_1 = 5^6 \bmod 23$

IT-21010

Find calculate 5^6:

$5^2 = 25 \equiv 2 \pmod{23}$

$5^4 = (5^2)^2 \equiv 2^2 = 4 \pmod{23}$

$5^6 = 5^4 \cdot 5^2 \equiv 4 \cdot 2 = 8 \pmod{23}$

So, $c_1 = 8$

compute c_2: $c_2 = m \cdot h^k \bmod p$

first find $h^k \bmod p$: $h^k = 8^6 \bmod 23$

$8^2 = 64 \equiv 18 \pmod{23}$

$8^4 \equiv 18^2 = 324 \equiv 2 \pmod{23}$

$8^6 = 8^4 \cdot 8^2 \equiv 2 \cdot 18 = 36 \equiv 13 \pmod{23}$

Now $c_2 = 10 \cdot 13 \bmod 23$

$c_2 = 130 \equiv 15 \pmod{23}$

The Elgamal ciphertext is

$(c_1, c_2) = (8, 15)$

So the encrypted message is

$(8, 15)$

01.IT-120101.0

3Q] Importance of Lightweight Cryptography for IoT

IoT devices often have limited processing power, limited memory, low energy availability, wireless communication. Because of these constraints, traditional cryptography like RSA or standard AES in full mode can be too resource-intensive.

Lightweight cryptography is designed to use smaller key sizes, require fewer CPU cycles, Minimize memory usage, consume less energy, still protect against eavesdropping, tampering and forgery.

IT is Important for IoT Because:

1) Security without sacrificing Performance, 2) Scalability 3) Longer battery life, 4) Protection in resource-constrained environment

Example Lightweight Encryption Algorithm

1) A block cipher designed for low resource devices
2) Block size 64 bits
3) Key sizes 80 or 128 bits
4) Uses a substitution-permutation Network (SPN)
5) Extremely low memory footprint

Other examples: SPECK, SIMON, LEA, KATAN and TinyAES

IT-21013

Q6] Three Common IoT-Specific Attacks & Mitigations

1) Firmware Hijacking: Attackers modify or replace the IoT devices firmware with malicious code, allowing them to take control or steal data. Unauthorized device control. Backdoors for persistent attacks. Use digitally signed firmware updates. Enable secure boot so only trusted firmware can run. Regularly update firmware from trusted sources.

2) Physical Tampering: Attackers physically access the IoT device to steal keys, modify hardware or attach debugging tools to extract sensitive info. Compromised cryptographic keys. Hardware damage or sabotage. Use tamper-resistant hardware. Store keys in secure elements or TPM chips. Place devices in physically secure locations whenever possible.

3) IoT Botnets: Malware infects IoT devices with weak/default passwords, adding them to a botnet for large-scale distributed denial of service (DDoS) attacks. Network outages due to massive traffic floods.

IT-2101O

Devices used in criminal activities without owner's knowledge. change default credentials, immediately after deployment. Implement strong authentication. Restrict unnecessary internet exposure. Keep firmware patched to close known vulnerabilities.

— 2 —